3月18日上午进度总结

1. 核心目标与完成情况

今天上午主要为大数计算器完善了三个重要的功能：

1. **除法四舍五入（Half-up）**：让除法在保留指定小数位时，能够像我们手算一样“满5进1”。
2. **支持科学计数法输入**：比如允许用户输入 `12e3` 或者 `1.23e-4`。
3. **统一固定精度输出**：只要设定了保留几位小数，不管是加减乘除，最后输出都会“对齐”，不够的补0，多余的四舍五入。

这三个功能都围绕着“让计算器的数字处理更加严谨、输出更整洁”这个目标。

---

2. 修改细节与新手讲解

2.1 除法四舍五入的实现

目的：以前的计算器可能截断小数，比如算到 `0.5` 但精度设置为0时直接变成 `0`。现在我们希望它变成 `1`。 **思路**：最稳妥的策略是“多算一位”。如果我们需要保留 `n` 位小数，那就先算出 `n+1` 位。看看第 `n+1` 位是不是 `>= 5`。如果是，就在第 `n` 位上加 1（也就是进位）。如果产生连锁进位（比如 `0.99` 加 `0.01` 变成 `1.00`），再把这个进位传导到整数部分。

修改的代码片段 (`calculator.c`): *注意：这部分代码位于 `bigint_div` 中。它抽取出了 `div_step` 函数来按步就算当前位，然后多算一位获取 `round_digit`。*

```
// 伪代码展示四舍五入逻辑（对应实际中的 increment_result_integer 和 bigint_div）
if (precision > 0) {
    int round_digit = 0;
    // ... 前面已经算出了 precision 位小数 ...

    // 多算1位用于判断是否进位
    if (strcmp(current_rem, "0") != 0) {
        round_digit = div_step(...);
    }

    // 进位规则 (half-up)
    if (round_digit >= 5) {
        int carry = 1;
        // 把最后一位小数往前加
        for (int i = wrote - 1; i >= 0 && carry; i--) {
            if (frac[i] < '9') { frac[i]++; carry = 0; }
            else { frac[i] = '0'; }
        }
        // 如果小数全变成0说明进位溢出到了整数部分
        if (carry) {
            increment_result_integer(result, MAX_BIGINT_LEN);
        }
    }
```

```
} else if (precision == 0 && strcmp(current_rem, "0") != 0) {
    // 精度为0的整数进位机制
    int d = div_step(...);
    if (d >= 5) increment_result_integer(result, MAX_BIGINT_LEN);
}
```

2.2 科学计数法输入的处理

目的：让用户可以输入 `1.2e3` 代替 `1200`，或者 `1e-3` 代替 `0.001`，提升输入便利性。 **思路**：为了不破坏你现在写好的加减乘除底层逻辑（底层只会算普通的数字符串），我们可以在**最开始读取输入（解析阶段）**，就把科学计数法**展开**成普通的数字。 例如，读到 `1.2e3`，我们知道 `1.2` 小数点后有1位，乘以 `10^3` 相当于给小数点向右移 3 位。因为 `1 - 3 = -2`，说明要在 `12` 后面补 2 个 `0`，展开成 `1200`。算好这笔账（`scale = frac_len - exp`），再用普通数字串传给大数计算逻辑，底层代码完全不用改。

修改的代码片段 (`calculator.c`):

```
// 新增的 parse_number_token 负责读取数字+科学计数法并展开
// 将 e/E 的输入转换成普通十进制
if (*cur == 'e' || *cur == 'E') {
    cur++;
    // 处理指数符号和数字...
    long exp = ...; // 例如 3 或 -2

    // scale = 原小数位数 - 指数
    long scale = frac_len - exp;

    // 展开字符串...
    if (scale <= 0) {
        // 小数点右移或补零（如 1.2e3 变成 1200）
    } else {
        // 小数点左移，可能会在前面插入零（如 123e-5 变成 0.00123）
    }
}
```

2.3 统一固定精度输出

目的：无论做了何种计算运算，如果设定了精度 `-p 2`，希望输出必须是 `xx.00` 格式对齐的，不省略尾随零。 **思路**：之前程序里有“去掉多余0”的代码，比如输入 `1.50` 会变成 `1.5`。既然现在我们要固定长度输出，这里就会发生矛盾（先去零，又得补零，逻辑重复）。因此，我们删掉原来的“去除末尾0”逻辑。在整个四则运算成功结束后，统一调用格式化函数 `format_fixed_precision`：

1. 先看它够不够 precision 位数，不够的在末尾补 `0` 补齐；
2. 如果有多余尾数则进行前面提到的四舍五入。 由于放到最后一步做截断/补齐，前面各种 `+-*/` 都是保持了最高精度在运算。

修改的代码片段 (`calculator.c`): 这里修改了 `main` 以及命令行/交互模式相关函数来接收精度参数：

```
// 1. main 函数中增加了对精度的解析
int precision = 6; // 默认精度
if (argc >= 3 && (strcmp(argv[1], "-p") == 0 || strcmp(argv[1], "--
precision") == 0)) {
    // 提取并设置 precision
    precision = (int)p;
}

// 2. 交互模式增加了切换精度的指令
if (strncmp(input, "-p ", 3) == 0) {
    // 动态修改精度
    precision = (int)p;
    continue;
}

// 3. 所有计算入口传递了 precision 参数
CalcResult evaluate_expression(const char *expr, int precision) {
    // 运算完毕后...
    // BIGINT_DIV 会用 precision:
    bigint_div(int1, int2, res.result, precision, &res.err);
    // 并且新增了统一格式化:
    // 最后返回前会使用 format_fixed_precision 强行补 0 或四舍五入
}

// 4. 去掉了 insert_decimal 原有去掉末尾0的冲突逻辑
// 之前的:
// while (final_len > 0 && temp[final_len - 1] == '0') final_len--;
// 现在只负责插入小数点，尾部是否填0交给最终步骤统一处理。
```

3. 架构总结

通过这三个功能的实现，我们梳理清楚了各个层的职责：

- **输入解析层**：专门处理用户的各种便捷写法（比如科学计数法），统一向后翻译成基础十进制字符。
- **大数运算核心**：心无旁骛，不管花里胡哨的输入输出，拿两串字符算加减乘除即可，提供最高的计算精度。
- **输出展示层**：在这里进行**格式对齐和四舍五入**。这样无论算除法还是乘法甚至加减，最终呈现都受统一调度。 这套“解耦”的逻辑非常利于之后扩展成基于AST（抽象语法树）的更复杂的计算器。